

Autor: Beanshie (nick)
Data: 14.03.2026
Uczelnia: Politechnika Krakowska

Optymalizacja zarządzania priorytetami QoS w sieciach - Kopiec Binarny i Tablica Nieuporządkowana

Spis treści

1	Wstęp do projektu	2
2	Metodologia badań i implementacja	4
2.1	Środowisko programistyczne i struktura danych	4
2.2	Charakterystyka scenariuszy testowych	4
2.3	Metodyka pomiarowa	5
3	Opis zaimplementowanych algorytmów	6
3.1	Naiwna Tablica Nieuporządkowana (Array)	6
3.2	Kopiec Binarny (Max-Heap)	6
4	Wyniki i analiza wykresów	8
4.1	Analiza drastycznych różnic wydajności (skala liniowa)	8
4.2	Weryfikacja teoretycznej złożoności i scenariuszy (skala log-log)	9
5	Podsumowanie i wnioski	10

1 Wstęp do projektu

Współczesne sieci teleinformatyczne opierają się na ciągłej i intensywnej wymianie milionów pakietów danych. Jednym z najbardziej narażonych na przeciążenia elementów tej architektury jest router – urządzenie brzegowe kierujące ruchem pomiędzy różnymi podsieciami. W sytuacji, gdy prędkość napływania pakietów (np. z szybkiej sieci lokalnej LAN) znacznie przewyższa możliwości przepustowe łącza wychodzącego (np. wolniejszego łącza WAN), powstaje zjawisko tzw. *wąskiego gardła* (ang. *bottleneck*).

Aby uniknąć bezpowrotnej utraty danych w momentach nagłego przeciążenia, routery wykorzystują bufor pamięciowe, w których pakiety oczekują w kolejce na wysłanie. Standardowe podejście typu FIFO (*First-In, First-Out*) okazuje się jednak całkowicie niewystarczające w nowoczesnym Internecie, gdzie usługi sieciowe mają drastycznie odmienne wymagania opóźnieniowe. Przykładowo, pobieranie dużego pliku w tle może zostać opóźnione o kilka sekund bez szkody dla użytkownika, podczas gdy opóźnienia w transmisji głosu (VoIP) lub strumieniowaniu wideo na żywo (IPTV) skutkują natychmiastową i krytyczną degradacją jakości usługi.

Rozwiązaniem tego problemu jest wdrożenie mechanizmów **Quality of Service (QoS)**. Pozwalają one na przypisanie każdemu pakietowi określonego priorytetu matematycznego, co gwarantuje, że ruch krytyczny z punktu widzenia działania systemu zostanie obsłużony w pierwszej kolejności, z pominięciem standardowej kolejki.

Celem niniejszego projektu jest implementacja, analiza wydajnościowa oraz weryfikacja założeń teoretycznych dla dwóch skrajnie różnych struktur danych, wykorzystywanych do zarządzania taką priorytetową kolejką pakietów:

- **Tablica Nieuporządkowana (Naiwna):** Reprezentująca najprostsze z możliwych rozwiązań, polegające na wrzucaniu pakietów na koniec bufora i każdorazowym liniowym przeszukiwaniu pamięci w celu znalezienia elementu o najwyższym priorytecie. Służy ona w badaniu jako punkt odniesienia (wzorzec negatywny), wykazujący pesymistyczną złożoność całkowitą $O(N^2)$.
- **Kopiec Binarny (Max-Heap):** Zaawansowana struktura drzewiasta, będąca branżowym standardem w implementacji kolejek priorytetowych w systemach wbudowanych. Wykorzystuje ona wewnętrzne mechanizmy operacji na drzewie binarnym (tzw. przesiewanie) w celu zagwarantowania sumarycznej złożoności na poziomie $O(N \log N)$.

Założenia projektowe i badawcze:

Aby rzetelnie ocenić efektywność obu struktur i ich zachowanie pod wpływem rosnącego obciążenia, zdefiniowano następujące założenia:

1. Pomiar wydajnościowy zostaną przeprowadzone w środowisku niskopoziomowym (C++) dla rosnących zestawów danych, w przedziale od 10 000 do 200 000 pakietów w kolejce.
2. Symulowany pakiet sieciowy zostanie zdefiniowany jako struktura o stałym rozmiarze 64 bajtów, co pozwoli na precyzyjne śledzenie zużycia pamięci operacyjnej (RAM) oraz optymalizację pod kątem pamięci podręcznej procesora.
3. Algorytmy zostaną poddane testom w trzech odrębnych scenariuszach dystrybucji priorytetów: w warunkach ruchu standardowego (*Uniform*), dla pesymistycznego przypadku strukturalnego z danymi posortowanymi (*Sorted*) oraz podczas symulacji nagłych zatorów sieciowych i ataków DDoS (*Burst*).

Zestawienie to pozwoli jednoznacznie odpowiedzieć na pytanie, w którym momencie narzut obliczeniowy tablicy dyskwalifikuje ją z profesjonalnych zastosowań telekomunikacyjnych na rzecz wysoce zoptymalizowanego kopca binarnego.

2 Metodologia badań i implementacja

W celu zapewnienia rzetelności, powtarzalności oraz wysokiej precyzji uzyskanych wyników, proces badawczy został podzielony na fazę generowania obciążenia sieciowego, fazę pomiarową w środowisku niskopoziomowym oraz fazę wizualizacji analitycznej.

2.1 Środowisko programistyczne i struktura danych

Główny silnik symulacyjny został napisany w języku **C++** (z wykorzystaniem standardowych kontenerów biblioteki STL). Wybór tego języka był podyktowany koniecznością wyeliminowania zjawiska odśmiecania pamięci (ang. *Garbage Collection*) oraz możliwością bezpośredniego zarządzania układem danych w pamięci RAM.

Kluczowym elementem optymalizacyjnym projektu była definicja pojedynczego pakietu danych. Został on zaprojektowany jako struktura (rekord) o z góry narzuconym, stałym rozmiarze:

- `uint32_t id` (4 bajty) – unikalny identyfikator pakietu.
- `uint32_t priority` (4 bajty) – priorytet QoS determinujący pozycję w kolejce.
- `std::array<uint8_t, 56> payload` (56 bajtów) – wyzerowany ładunek użyteczny symulujący rzeczywiste dane sieciowe.

Sumaryczny rozmiar struktury wynosi dokładnie **64 bajty**. Wartość ta nie jest przypadkowa – odpowiada ona dokładnie rozmiarowi pojedynczej linii pamięci podręcznej (ang. *Cache Line*) w nowoczesnych procesorach architektury x86-64. Dzięki temu, pobieranie pakietów z pamięci RAM do pamięci L1 procesora następuje w sposób optymalny, minimalizując zjawisko tzw. *cache misses*.

Obliczenie teoretycznego zużycia pamięci operacyjnej (RAM) dla obu struktur sprowadza się do prostej zależności matematycznej:

$$RAM = N \times \text{sizeof}(\text{Packet})$$

Gdzie N to liczba pakietów. Ponieważ zarówno naiwna tablica, jak i kopiec binarny zostały oparte na płaskim kontenerze `std::vector`, zużywają one identyczną ilość pamięci per pakiet (brak narzutu na wskaźniki węzłów, jak miałyoby to miejsce w klasycznych drzewach). Dla maksymalnej próby badawczej ($N = 200\,000$), struktura zajmuje w pamięci zaledwie około 12.2 MB ($200\,000 \times 64$ bajty).

2.2 Charakterystyka scenariuszy testowych

Aby zbadać zachowanie algorytmów w różnych warunkach obciążenia sieci, każdy z nich przetestowano w trzech zróżnicowanych scenariuszach przypisywania priorytetów (od 1 do 100):

- **Uniform (Ruch jednostajny):** Pakiety otrzymują całkowicie losowe priorytety o rozkładzie równomiernym. Jest to scenariusz bazowy, symulujący standardowy ruch w poprawnie funkcjonującej sieci.
- **Sorted (Najgorszy przypadek / Worst-case):** Pakiety trafiają do kolejki posortowane od priorytetu najniższego do najwyższego. Stanowi to pułapkę na strukturę kopca binarnego – zmusza algorytm wstawiania do wykonania maksymalnej liczby rotacji (przesiewania w górę) przy każdym nowym elemencie.

- **Burst (Atak DDoS / Zator sieciowy):** Scenariusz kryzysowy zaimplementowany przy użyciu rozkładu probabilistycznego. Z prawdopodobieństwem 90% generowane są pakiety o krytycznie wysokim priorytecie (95-100), a jedynie 10% stanowi ruch standardowy (1-100). Symuluje to sytuację, w której setki użytkowników naraz żądają transmisji wideo, tworząc gigantyczny tłok na najwyższych priorytetach.

2.3 Metodyka pomiarowa

Do precyzyjnego pomiaru zużycia czasu procesora (CPU) wykorzystano bibliotekę `<chrono>`, a dokładnie zegar wysokiej rozdzielczości `std::chrono::high_resolution_clock`. Wyniki rejestrowano z dokładnością do mikrosekund (μs).

Kluczowym założeniem badawczym było odseparowanie czasu generowania danych od czasu operacji na strukturach. Stoper uruchamiany był dopiero w momencie, gdy zbiór testowy pakietów znajdował się już w pamięci podręcznej. Pomiar obejmował wyłącznie zsumowany czas operacji wstawienia całego zbioru do kolejki priorytetowej (`push`), a następnie wyciągnięcia z niej wszystkich elementów, jeden po drugim (`pop`). Czas operacji wejścia/wyjścia, takich jak zapis pomiarów do plików `.csv`, został celowo wyłączony z pomiaru.

3 Opis zaimplementowanych algorytmów

W niniejszym rozdziale przedstawiono teoretyczną analizę oraz mechanikę działania dwóch struktur danych, które zaimplementowano w celu symulacji kolejki priorytetowej QoS. Oba algorytmy zostały opatrzone ujednoliconym pseudokodem, obrazującym operacje dodawania nowego pakietu (**push**) oraz pobierania pakietu o najwyższym priorytecie (**pop**).

3.1 Naiwna Tablica Nieuporządkowana (Array)

Rozwiązanie to opiera się na wykorzystaniu płaskiej, dynamicznej tablicy (wektora), do której pakiety sieciowe są po prostu dopisywane na sam koniec. Jest to implementacja skrajnie asymetryczna pod względem wydajnościowym.

Operacja wstawienia (**push**) jest natychmiastowa i posiada złożoność $O(1)$. Jednakże, aby router mógł wysłać najważniejszy pakiet, operacja pobrania (**pop**) wymusza liniowe przeszukanie całej zawartości bufora w poszukiwaniu elementu o najwyższym priorytecie. Po znalezieniu maksimum, algorytm zamienia je miejscami z ostatnim elementem w tablicy i usuwa, aby uniknąć kosztownego przesuwania pozostałych danych.

Ponieważ dla zbioru N pakietów musimy N -krotnie wykonać operację przeszukiwania (której koszt maleje liniowo, ale wciąż zależy od N), całkowita złożoność procesu zarządzania kolejką w tym przypadku jest kwadratowa i wynosi $O(N^2)$.

Algorithm 1 Naiwna Tablica – Wstawianie i Pobieranie

```
procedure push(packet)  
  array.append(packet)  
end procedure  
  
procedure pop()  
  maxIndex  $\leftarrow$  0  
  for  $i = 1$  to array.size()  $- 1$  do  
    if array[ $i$ ].priority > array[maxIndex].priority then  
      maxIndex  $\leftarrow$   $i$   
    end if  
  end for  
  maxPacket  $\leftarrow$  array[maxIndex]  
  swap(array[maxIndex], array[array.size()  $- 1$ ])  
  array.removeLast()  
  return maxPacket  
end procedure
```

3.2 Kopiec Binarny (Max-Heap)

Kopiec binarny to zaawansowana struktura danych, która logicznie przyjmuje formę kompletnego drzewa binarnego, ale fizycznie przechowywana jest w płaskiej tablicy. Relacje między węzłami wyliczane są za pomocą prostych indeksów matematycznych: dla węzła o indeksie i , jego rodzic znajduje się pod indeksem $(i - 1)/2$, a dzieci pod $2i + 1$ oraz $2i + 2$. Główna zasada (własność kopca typu Max) mówi, że priorytet każdego rodzica musi być większy lub równy priorytetom jego dzieci. Dzięki temu pakiet o najwyższym priorytecie zawsze znajduje się w korzeniu drzewa (pod indeksem 0).

Dodanie nowego pakietu (**push**) polega na wrzuceniu go na koniec tablicy (jako najmłodszy liść), a następnie przywróceniu własności kopca poprzez mechanizm bąbelkowania w górę (**heapifyUp**). Pakiet zamienia się miejscami ze swoim rodzicem, dopóki nie trafi na kogoś ważniejszego od siebie.

Podczas pobierania (**pop**), korzeń jest usuwany (wysyłany przez router), a na jego miejsce wstawiany jest ostatni liść. Następnie struktura naprawia się, przesiewając ten element w dół drzewa (**heapifyDown**).

Ponieważ wysokość drzewa binarnego wynosi $\log_2(N)$, liczba maksymalnych porównań dla pojedynczej operacji to $\log N$. Dla N pakietów sumaryczna złożoność algorytmu to wysoce efektywne $O(N \log N)$.

Algorithm 2 Kopiec Binarny (Max-Heap) – Dodawanie i Przesiewanie

```

procedure push(packet)
  heap.append(packet)
  heapifyUp(heap.size() - 1)
end procedure

procedure heapifyUp(index)
  while index > 0 do
    parent ← (index - 1)/2
    if heap[index].priority > heap[parent].priority then
      swap(heap[index], heap[parent])
      index ← parent
    else
      break
    end if
  end while
end procedure

```

Algorithm 3 Kopiec Binarny (Max-Heap) – Pobieranie (bez funkcji heapifyDown)

```

procedure pop()
  maxPacket ← heap[0]
  heap[0] ← heap[heap.size() - 1]
  heap.removeLast()
  heapifyDown(0)           // Przesiewanie w dół (analogiczne do heapifyUp)
  return maxPacket
end procedure

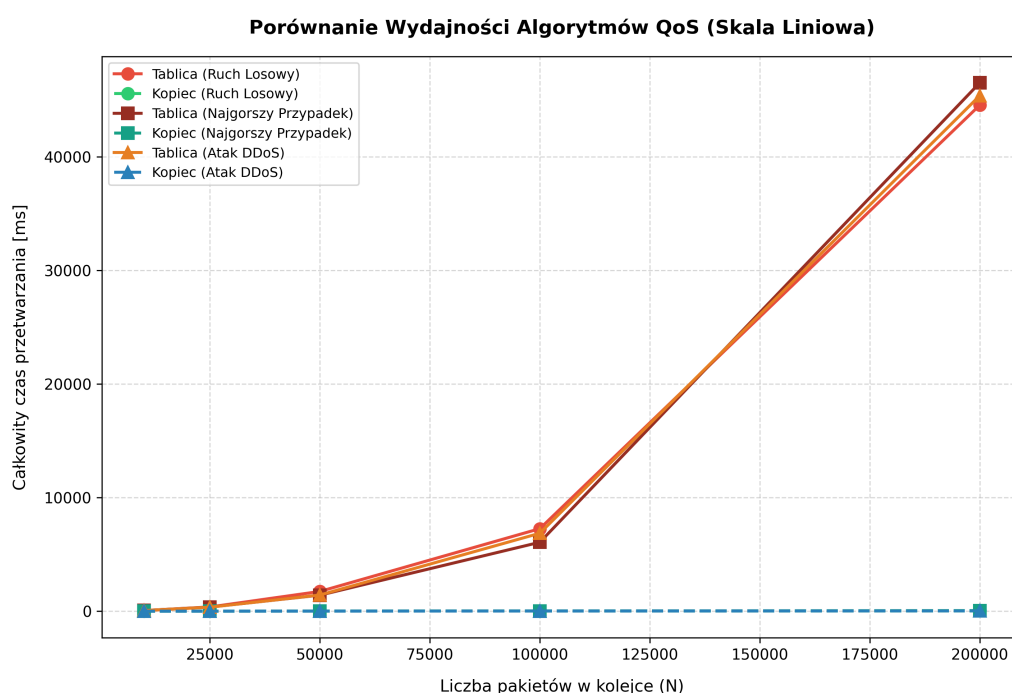
```

4 Wyniki i analiza wykresów

W niniejszym rozdziale zaprezentowano graficzną wizualizację danych zebranych podczas testów wydajnościowych obu struktur. Aby w pełni zobrazować przepaść technologiczną dzielącą Naiwną Tablicę od Kopca Binarnego, analizę przeprowadzono w dwóch ujęciach: na standardowej skali liniowej (ukazującej drastyczny wzrost czasu rzeczywistego) oraz na podwójnej skali logarytmicznej (pozwalającej na matematyczną weryfikację rzędu złożoności O).

4.1 Analiza drastycznych różnic wydajności (skala liniowa)

Pierwszy wykres przedstawia zależność całkowitego czasu przetwarzania kolejki (w mikrosekundach) od liczby pakietów, z wykorzystaniem klasycznej, liniowej skali osi Y.



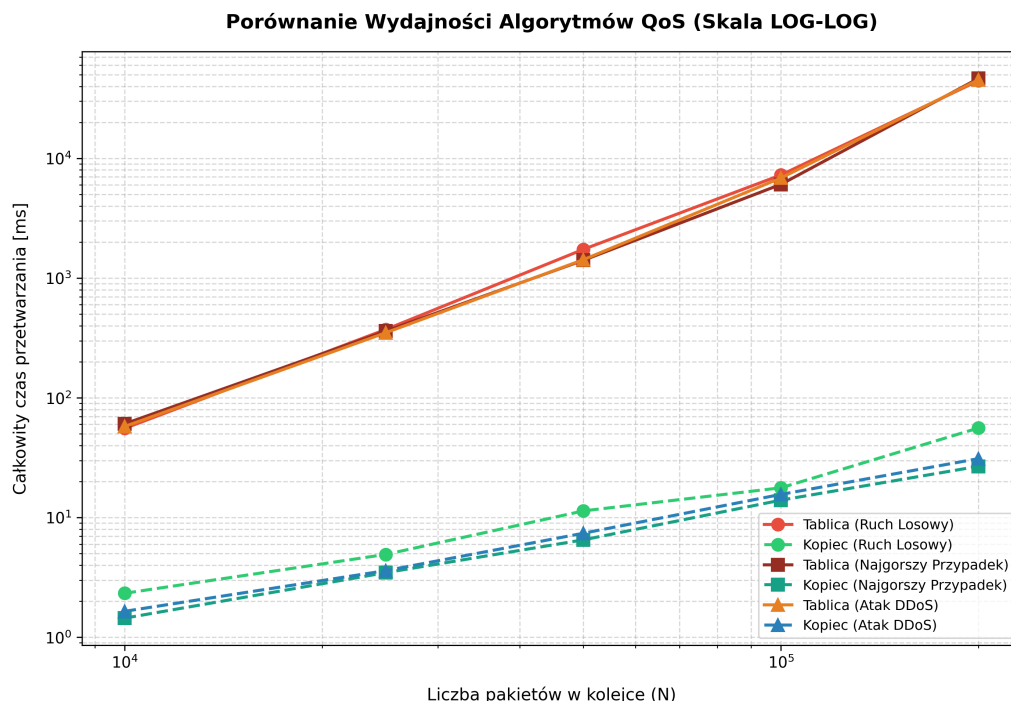
Rysunek 1: Porównanie czasu zarządzania pakietami w skali liniowej.

Wykres ten stanowi bezwzględny dowód na nieefektywność naiwnej implementacji w środowisku sieciowym. Krzywa reprezentująca Tablicę Nieuporządkowaną przybiera kształt stromej paraboli. Przy obciążeniu rzędu 200 000 pakietów, czas potrzebny na odnalezienie i wysłanie priorytetowych danych drastycznie rośnie. W rzeczywistym środowisku telekomunikacyjnym taki narzut czasowy doprowadziłby do całkowitego "zamrożenia" routera, przepełnienia buforów wejściowych i w efekcie do masowego odrzucania (ang. *dropping*) kolejnych pakietów.

Z kolei czas wykonania algorytmu Kopca Binarnego na tym samym wykresie wydaje się być niemal płaską linią, bliską osi X. Sugeruje to, że dla tej skali danych, narzut obliczeniowy wprowadzany przez operacje na drzewie binarnym jest z perspektywy procesora niemal pomijalny.

4.2 Weryfikacja teoretycznej złożoności i scenariuszy (skala log-log)

Aby dokładnie zbadać zachowanie Kopca Binarnego oraz zweryfikować założenia analityczne (notację O), te same dane pomiarowe naniesiono na wykres o podwójnej skali logarytmicznej ($\log\text{-}\log$). W takim ujęciu krzywe potęgowe i logarytmiczne transformują się w linie proste o różnym kącie nachylenia.



Rysunek 2: Weryfikacja rzędu złożoności algorytmów oraz analiza scenariuszy ruchu sieciowego (skala log-log).

Analiza powyższego wykresu dostarcza dwóch kluczowych wniosków:

- Potwierdzenie rzędów złożoności:** Proste reprezentujące Naiwną Tablicę charakteryzują się bardzo stromym kątem nachylenia, co jednoznacznie potwierdza jej przynależność do klasy złożoności kwadratowej $O(N^2)$. Znacznie łagodniejsze nachylenie linii Kopca Binarnego dowodzi jego log- liniowej złożoności $O(N \log N)$.
- Odporność na zmienne scenariusze sieciowe:** Wykres uwzględnia podział na różne scenariusze ruchu (Uniform, Burst, Sorted). Naiwna Tablica wykazuje identyczną, tragiczną wydajność niezależnie od scenariusza, ponieważ i tak musi zawsze przeszukać cały bufor. W przypadku Kopca Binarnego możemy zaobserwować minimalne rozwarstwienie wyników. Kopek działa najszybciej dla losowego ruchu (*Uniform*) oraz ataków DDoS (*Burst*), gdzie z uwagi na dużą ilość pakietów o najwyższym priorytecie (95-100), bąbelkowanie w górę zatrzymuje się bardzo wcześnie. Przypadek posortowany (*Sorted*) generuje minimalnie większy narzut czasowy (najgorszy przypadek wstawiania), jednak wciąż z łatwością utrzymuje stabilny trend $O(N \log N)$, gwarantując bezpieczeństwo i płynność działania routera.

5 Podsumowanie i wnioski

Przeprowadzona w ramach niniejszego projektu kompleksowa analiza wydajnościowa pozwoliła na weryfikację teoretycznych założeń dotyczących struktur danych w krytycznym środowisku telekomunikacyjnym. Na podstawie zebranych danych empirycznych oraz wygenerowanych modeli graficznych sformułowano następujące wnioski końcowe:

1. **Zbieżność teorii z praktyką:** Zastosowanie podwójnej skali logarytmicznej w analizie wyników bezspornie potwierdziło przynależność badanych algorytmów do ich teoretycznych klas złożoności. Naiwna Tablica wykazała wzrost kwadratowy $O(N^2)$, podczas gdy Kopiec Binarny utrzymał stabilny trend log-liniowy $O(N \log N)$. Udowadnia to poprawność zaimplementowanych w języku C++ mechanizmów zarządzania pamięcią.
2. **Dyskwalifikacja algorytmów kwadratowych w systemach QoS:** Wykresy o skali liniowej obnażyły drastyczne słabości Tablicy Nieuporządkowanej. Konieczność każdorazowego, liniowego przeszukiwania bufora sprawia, że algorytm ten "zatyka się" już przy kilkudziesięciu tysiącach elementów. W rzeczywistym routerze doprowadziłoby to do paraliżu urządzenia i masowego odrzucania ruchu (w tym krytycznych pakietów VoIP/Wideo).
3. **Odporność Kopca Binarnego na anomalie sieciowe:** Testy z wykorzystaniem dystrybucji prawdopodobieństwa (scenariusz *Burst*) udowodniły, że Kopiec Binarny radzi sobie doskonale w sytuacjach nagłego przeciążenia (np. ataki DDoS, piki popularności usług streamingowych). Nawet przy wymuszeniu najgorszego przypadku strukturalnego (scenariusz *Sorted*), operacje przesiewania gwarantują, że urządzenie zachowa płynność działania i poprawnie wyegzekwuje priorytety QoS.
4. **Optymalizacja sprzętowa (Cache / RAM):** Projekt udowodnił, że świadome zarządzanie układem danych w pamięci ma kluczowe znaczenie. Zamknięcie struktury pakietu w równej 64-bajtowej linii pamięci podręcznej (*Cache Line*) zminimalizowało narzut procesora. Jednocześnie wykazano, że struktura wirtualnego drzewa binarnego (oparta o płaski wektor zamiast wskaźników) jest wysoce energooszczędna – utrzymanie w kolejce maksymalnej badanej próby 200 000 pakietów zaalokowało zaledwie około 12.2 MB pamięci operacyjnej.

Podsumowując, badanie empiryczne jednoznacznie potwierdza, że do zarządzania priorytetowym ruchem sieciowym QoS nie można stosować prymitywnych struktur przeszukiwanych liniowo. Optymalnym, sprawdzonym i całkowicie skalowalnym rozwiązaniem pozostaje Kopiec Binarny, który przy minimalnym zużyciu pamięci RAM zapewnia logarytmiczny czas dostępu, chroniąc infrastrukturę sieciową przed zatorami.